fastmcp_observability_cal...
Zip Archive

Using this code, I want to add a streaming response so that it can tell what tools its calling and its general flow. How do I do that?

I'll inspect the uploaded project structure first, then map the smallest streaming change onto your existing FastAPI/backend/frontend flow. I'll focus on streaming tool events like "calling math_add" plus normal text chunks, not a full architecture rewrite.

I found your app is already in a good position for this: `RequestObserver` sees real LangChain tool and LLM callbacks, so the smallest change is to let that observer push events into a stream. Then add a new `/chat/stream` endpoint while keeping `/chat` unchanged.

Thought for 2m 25s

I added the streaming version as a **minimal patch** to your uploaded project here:

Download the streaming patch ZIP

The approach is:

1. Keep your existing `/chat` endpoint unchanged.
2. Add a new `/chat/stream` endpoint.
3. Reuse your existing `RequestObserver`, but let it optionally push events into an async queue.
4. Stream those events to the client as Server-Sent Events.

This fits your current architecture because your `RequestObserver` already sees the important runtime events: LLM starts, tool starts, tool ends, tool errors. LangChain's own streaming docs describe streaming agent progress, LLM tokens, custom updates, and tool activity as the right pattern for responsive agent UX. Docs by LangCh... FastAPI's SSE docs describe streaming with `text/event-stream`, where each event is sent as a small event block over HTTP. FastAPI

The patched files are:

```
backend/observability/request_observer.py
backend/agent.py
backend/session_store.py
backend/main.py
```

The new endpoint is:

```
POST /chat/stream
```

You can test it with:

Bash

```bash
curl -N -X POST http://127.0.0.1:8000/chat/stream \
  -H "Content-Type: application/json" \
  -d '{"session_id":"demo-stream","message":"What is 17 + 28? Use a tool."}'
```

You should see events like:

```
event: flow
data: {"type":"flow","message":"Waiting for the session lock.",...}

event: llm_start
data: {"type":"llm_start","message":"Starting chat_model call #1",...}

event: tool_start
data: {"type":"tool_start","message":"Calling tool: math_add","tool_name":"math_add",.

event: tool_end
data: {"type":"tool_end","message":"Finished tool: math_add","tool_name":"math_add",..

event: final
data: {"type":"final","reply":"17 + 28 = 45",...}

event: done
data: {"type":"done","success":true,...}
```

For a frontend using `fetch`, you would consume it roughly like this:

JavaScript

```
const response = await fetch("http://127.0.0.1:8000/chat/stream", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    session_id: "demo-stream",
    message: "What is 17 + 28? Use a tool."
  })
});

const reader = response.body.getReader();
const decoder = new TextDecoder();

let buffer = "";

while (true) {
  const { value, done } = await reader.read();
  if (done) break;

  buffer += decoder.decode(value, { stream: true });

  const events = buffer.split("\n\n");
  buffer = events.pop() || "";

  for (const rawEvent of events) {
    const lines = rawEvent.split("\n");
    const eventLine = lines.find(line => line.startsWith("event: "));
    const dataLine = lines.find(line => line.startsWith("data: "));

    if (!eventLine || !dataLine) continue;

    const eventType = eventLine.replace("event: ", "");
    const data = JSON.parse(dataLine.replace("data: ", ""));

    if (eventType === "tool_start") {
      console.log("Tool starting:", data.tool_name);
    }

    if (eventType === "tool_end") {
      console.log("Tool finished:", data.tool_name);
    }

    if (eventType === "final") {
      console.log("Assistant reply:", data.reply);
    }
  }
}
```

One important note: this streams **agent progress and tool activity**, then sends the final reply at the end. It does not yet stream every final-answer token. That is intentional because your current observability code is callback-based and already cleanly supports tool/flow streaming. Token streaming can be added afterward, but it is slightly noisier because intermediate LLM calls may include planning/tool-call text, not just the final answer.